OAVS TGT COMPUTER TEACHER EXAMINATION

# Multithreading

Exam-Focused MCQ Question Bank

**SYLLABUS COVERED**

- Multithreading Concept
- Thread Life Cycle

| Section | Title | Questions | Q. Nos. |
|---|---|---|---|
| 1 | Multithreading Concept | 16 | Q1 – Q16 |
| 2 | Thread Life Cycle | 9 | Q17 – Q25 |
| 3 | Thread States and Transitions | 8 | Q26 – Q33 |
| 4 | Scenario / Diagram-Based Questions | 8 | Q34 – Q41 |
| 5 | Important & Tricky Questions | 8 | Q42 – Q49 |
| 6 | Mixed OAVS TGT Practice Set | 10 | Q50 – Q59 |
| 7 | Supplementary Practice Set | 21 | Q60 – Q80 |
| | **Total** | **80** | **Q1 – Q80** |

**How to use:** Attempt each question first, then check the green answer strip and the short explanation. Each card shows: question number | question type | topic. A complete answer key is given on the last page.

SECTION 1

## Multithreading Concept

16 Questions

Definitions, characteristics, terminology, advantages

**Q1 | Definition | Multithreading**

**Multithreading is best defined as:**

**A.** Execution of two or more separate programs in different processes only

**B.** Sequential execution of every instruction through a single flow of control

**C.** Concurrent execution of two or more threads within a single process

**D.** Compilation of source code by several compilers at the same time

**✔ Ans: C — Concurrent execution of two or more threads within a single process**

**Explanation:** Multithreading lets multiple threads (independent paths of execution) run concurrently inside one process.

**Q2 | Concept | Multithreading**

**A thread is commonly described as:**

**A.** A heavyweight program that owns a separate address space

**B.** A file that stores compiled bytecode of a class

**C.** A lightweight unit of execution within a process

**D.** A hardware component that fetches and decodes instructions

**✔ Ans: C — A lightweight unit of execution within a process**

**Explanation:** A thread is the smallest unit of execution; it is called lightweight because it lives inside a process and shares its resources.

**Q3 | Characteristic | Multithreading**

**Which of the following is shared by all threads of the same process?**

**A.** The register values saved for each thread

**B.** The heap memory and code of the process

**C.** The private call stack owned by each thread

**D.** The program counter maintained by each thread

**✔ Ans: B — The heap memory and code of the process**

**Explanation:** Each thread has its own stack, program counter and registers; code and heap data belong to the process and are shared.

**Q4 | Terminology | Multithreading**

**Multithreading is an example of:**

**A.** Process-based multitasking

**B.** Batch processing

**C.** Spooling of print jobs

**D.** Thread-based multitasking

**✔ Ans: D — Thread-based multitasking**

**Explanation:** Thread-based multitasking runs parts of the same program concurrently; process-based multitasking runs separate programs.

**Q5 | Characteristic | Multithreading**

**Compared with processes, threads are generally:**

**A.** Cheaper to create and cheaper to switch between

**B.** Unable to share any data with each other

**C.** Completely independent of the process that holds them

**D.** Costlier to create because each thread needs its own address space

**✓ Ans: A — Cheaper to create and cheaper to switch between**

**Explanation:** Threads share the process's address space, so creation and context switching involve less overhead.

**Q6 | Purpose | Multithreading**

**The primary purpose of multithreading is to:**

**A.** Guarantee faster execution of every program on a single-core CPU

**B.** Remove the need for an operating system to manage running tasks

**C.** Reduce the size of the compiled program file on disk

**D.** Perform several tasks concurrently for better CPU utilization and responsiveness

**✓ Ans: D — Perform several tasks concurrently for better CPU utilization and responsiveness**

**Explanation:** Multithreading keeps the CPU busy and the application responsive by overlapping tasks (e.g., computation and waiting).

**Q7 | Advantage | Multithreading**

**Which of the following is NOT an advantage of multithreading?**

**A.** Economical sharing of resources among related tasks

**B.** Program design becomes simpler and its behaviour fully predictable

**C.** Better utilization of idle CPU time during waiting

**D.** Improved responsiveness of interactive applications

**✓ Ans: B — Program design becomes simpler and its behaviour fully predictable**

**Explanation:** Multithreading actually makes design, testing and debugging more complex, so simplicity is not an advantage.

**Q8 | Terminology | Multithreading**

**Tasks that are executing at the very same instant on different CPU cores are said to run in:**

**A.** Parallel

**B.** Time-sliced mode

**C.** Serialized mode

**D.** Context-switched mode

**✓ Ans: A — Parallel**

**Explanation:** Parallelism needs multiple cores; time slicing only interleaves tasks on a core to give an illusion of simultaneity.

**Q9 | Terminology | Multithreading**

**In multithreading, a *context switch* refers to:**

**A.** Terminating a thread and immediately starting a fresh one in its place

**B.** Saving the state of one thread and loading the state of another so the CPU can change over

**C.** Permanently moving all of a thread's data from RAM to disk storage

**D.** Converting a running thread into a completely separate process

**✔ Ans: B — Saving the state of one thread and loading the state of another so the CPU can change over**

**Explanation:** The CPU stores the current thread's state (registers, program counter) and restores another's; this switching has a cost.

**Q10 | Concept | Multithreading**

**On a single-core CPU, several threads appear to run simultaneously because:**

**A.** Threads are always executed on the graphics card instead of the CPU

**B.** The compiler merges all threads into one single instruction stream

**C.** The CPU switches rapidly among the threads giving each a short time slice

**D.** Each thread is automatically given its own physical core by the OS

**✔ Ans: C — The CPU switches rapidly among the threads giving each a short time slice**

**Explanation:** Rapid switching (time slicing) creates concurrency without true parallelism on one core.

**Q11 | Scenario | Multithreading**

**A word processor lets you keep typing while it checks spelling and auto-saves the document in the background. Which programming concept makes this possible?**

**A.** Multithreading

**B.** Recursion

**C.** Type casting

**D.** Method overloading

**✔ Ans: A — Multithreading**

**Explanation:** Separate threads handle typing, spell-check and auto-save concurrently.

**Q12 | Statement-based | Multithreading**

**Consider the statements:**
**I. Threads of a process share the same memory space.**
**II. Context switching among threads is generally cheaper than among processes.**
**III. Every thread has a completely independent address space.**
**Which statements are correct?**

**A.** I, II and III

**B.** I and III only

**C.** II and III only

**D.** I and II only

**✔ Ans: D — I and II only**

**Explanation:** Threads share the process address space, so III is false.

**Q13 | Assertion–Reason | Multithreading**

**Assertion (A): Multithreading improves the responsiveness of GUI applications.**
**Reason (R): A long-running task can execute in a background thread while the user-interface thread stays active.**

**A.** A is true, but R is false

**B.** A is false, but R is true

**C.** Both A and R are true, and R is the correct explanation of A

**D.** Both A and R are true, but R is not the correct explanation of A

**✔ Ans: C — Both A and R are true, and R is the correct explanation of A**

**Explanation:** The background thread keeps the interface from freezing, which directly explains the improved responsiveness.

**Q14 | Assertion–Reason | Multithreading**

**Assertion (A): Using multithreading always makes every program run faster.**
**Reason (R): Context switching and managing multiple threads add overhead.**

**A.** A is true, but R is false

**B.** Both A and R are true, and R is the correct explanation of A

**C.** A is false, but R is true

**D.** Both A and R are true, but R is not the correct explanation of A

**✔ Ans: C — A is false, but R is true**

**Explanation:** Overhead can cancel the benefit (e.g., CPU-bound work on a single core), so A is false while R is true.

**Q15 | Match | Multithreading**

**Match List I with List II:**
**List I: 1. Thread 2. Multitasking 3. Context switch 4. Concurrency**
**List II: a. Several tasks/programs managed by the OS at the same time b. Lightweight unit of execution inside a process c. Saving one thread's state and loading another's d. Multiple tasks making progress in overlapping time periods**

**A.** 1-b, 2-a, 3-c, 4-d

**B.** 1-a, 2-b, 3-c, 4-d

**C.** 1-b, 2-a, 3-d, 4-c

**D.** 1-c, 2-b, 3-a, 4-d

**✔ Ans: A — 1-b, 2-a, 3-c, 4-d**

**Explanation:** Thread = lightweight unit (b); multitasking = many tasks by OS (a); context switch = save/load state (c); concurrency = overlapping progress (d).

**Q16 | Concept | Multithreading**

**Which of the following is a recognised disadvantage of multithreading?**

**A.** It always reduces the responsiveness of the application

**B.** Threads cannot share any data with one another at all

**C.** Context-switching overhead and increased program complexity

**D.** Every thread requires a separate copy of the program code

**✔ Ans: C — Context-switching overhead and increased program complexity**

**Explanation:** Extra overhead and harder design/debugging are the usual drawbacks.

SECTION 2

## Thread Life Cycle

9 Questions

States of a thread from creation to termination

**Q17 | Life Cycle | Thread Life Cycle**

**The life cycle of a thread refers to:**

**A.** The number of processors that are allotted to a thread

**B.** The different states a thread passes through from its creation to its termination

**C.** The order in which the threads are written in the source code of a program

**D.** The amount of memory a thread occupies while it is running

**✔ Ans: B — The different states a thread passes through from its creation to its termination**

**Explanation:** A thread moves through states such as New, Runnable, Waiting and Terminated during its lifetime.

**Q18 | Terminology | Thread Life Cycle**

**How many states are defined in Java's *Thread.State* enumeration?**

**A.** Six

**B.** Seven

**C.** Four

**D.** Five

**✔ Ans: A — Six**

**Explanation:** NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING and TERMINATED.

**Q19 | State | Thread Life Cycle**

**A thread object has been created but start() has not yet been invoked. The thread is in the:**

**A.** Runnable state

**B.** Blocked state

**C.** Terminated state

**D.** New state

**✔ Ans: D — New state**

**Explanation:** A thread stays in the New state until start() is called; it is not yet alive.

**Q20 | Transition | Thread Life Cycle**

**Which call moves a thread from the New state to the Runnable state?**

**A.** resume()

**B.** run()

**C.** init()

**D.** start()

**✔ Ans: D — start()**

**Explanation:** start() makes the thread eligible for scheduling; the thread then executes its run() method.

**Q21 | State | Thread Life Cycle**

**Which is the last state in the life cycle of a thread?**

**A.** Runnable
**B.** Blocked/Waiting
**C.** Terminated (Dead)
**D.** Timed Waiting

✓ **Ans: C — Terminated (Dead)**

**Explanation:** After run() completes, the thread is Terminated and its life cycle ends.

**Q22 | State | Thread Life Cycle**

**A thread in the Runnable state is one that is:**

**A.** Waiting for a specified period of time to elapse
**B.** Created but not yet started by the program
**C.** Ready to run, waiting for CPU time or already executing
**D.** Finished with its run() method completely

✓ **Ans: C — Ready to run, waiting for CPU time or already executing**

**Explanation:** Runnable covers a thread eligible for the CPU, whether or not it currently holds it.

**Q23 | State | Thread Life Cycle**

**A thread executes sleep(2000). During these two seconds it is in the:**

**A.** Waiting state with no time limit
**B.** Timed Waiting state
**C.** New state
**D.** Terminated state

✓ **Ans: B — Timed Waiting state**

**Explanation:** sleep() gives a fixed time limit, so the thread is in TIMED_WAITING, not plain WAITING.

**Q24 | State | Thread Life Cycle**

**A thread in the Blocked/Waiting (non-runnable) state is:**

**A.** Executing on the CPU at its maximum possible speed
**B.** Not yet started by the program at all
**C.** Dead and unable to be used again by the program
**D.** Alive but temporarily inactive and not eligible for the CPU

✓ **Ans: D — Alive but temporarily inactive and not eligible for the CPU**

**Explanation:** It will return to Runnable once the awaited condition occurs.

**Q25 | Transition | Thread Life Cycle**

**A thread enters the Terminated state when:**

**A.** It is selected by the thread scheduler for execution

**B.** Its run() method completes or an uncaught exception ends it

**C.** start() is invoked on it for the first time

**D.** It enters the sleeping condition for some time

**✓ Ans: B — Its run() method completes or an uncaught exception ends it**

**Explanation:** Normal completion of run() or an uncaught exception ends the thread.

SECTION 3

## Thread States and Transitions

8 Questions

State identification and movement between states

**Q26 | Transition | Thread Life Cycle**

**Which of the following transitions is NOT possible in the thread life cycle?**

**A.** Running → Blocked/Waiting

**B.** Terminated → Runnable

**C.** New → Runnable

**D.** Blocked/Waiting → Runnable

**✓ Ans: B — Terminated → Runnable**

**Explanation:** A terminated thread can never be revived.

**Q27 | Transition | Thread Life Cycle**

**When the thread scheduler picks a ready thread and gives it the CPU, the thread moves from:**

**A.** Running to Terminated

**B.** Blocked to Running

**C.** Runnable to Running

**D.** New to Runnable

**✓ Ans: C — Runnable to Running**

**Explanation:** Dispatching by the scheduler is the Runnable → Running transition.

**Q28 | Transition | Thread Life Cycle**

**From the Running state, a thread can move directly to:**

**A.** New, Runnable or Terminated

**B.** Runnable, Blocked/Waiting or Terminated

**C.** New, Blocked/Waiting or Runnable

**D.** Only Blocked/Waiting or New

**✓ Ans: B — Runnable, Blocked/Waiting or Terminated**

**Explanation:** It may be pre-empted (Runnable), wait for I/O or sleep (Blocked/Waiting), or finish (Terminated). It can never return to New.

**Q29 | State | Thread Life Cycle**

**Waiting and Timed Waiting states differ in that:**

**A.** Timed Waiting has a specified maximum waiting time, whereas Waiting has none

**B.** A Waiting thread is dead while a Timed Waiting thread is alive

**C.** A Timed Waiting thread uses the CPU while a Waiting thread does not

**D.** Waiting can be entered only from the New state

**✓ Ans: A — Timed Waiting has a specified maximum waiting time, whereas Waiting has none**

**Explanation:** Both are non-runnable; Timed Waiting ends automatically when its time expires.

**Q30 | State | Thread Life Cycle**

**Which group contains only states in which a started, live thread cannot be given the CPU?**

**A.** Blocked, Waiting, Timed Waiting

**B.** Waiting, Runnable, New

**C.** New, Runnable, Terminated

**D.** Runnable, Blocked, Terminated

**✓ Ans: A — Blocked, Waiting, Timed Waiting**

**Explanation:** Runnable threads are eligible for the CPU; Blocked/Waiting/Timed Waiting threads are not.

**Q31 | Match | Thread Life Cycle**

**Match the event with the resulting transition:**
**Event: 1. Running thread requests I/O 2. Time slice expires 3. run() ends 4. Sleep time elapses**
**Transition: a. Running → Runnable b. Running → Blocked/Waiting c. Running → Terminated d. Timed Waiting → Runnable**

**A.** 1-a, 2-b, 3-c, 4-d

**B.** 1-c, 2-a, 3-b, 4-d

**C.** 1-b, 2-a, 3-c, 4-d

**D.** 1-b, 2-a, 3-d, 4-c

**✓ Ans: C — 1-b, 2-a, 3-c, 4-d**

**Explanation:** I/O makes the thread wait; time slice expiry returns it to Runnable; run() completion terminates it; end of sleep makes it Runnable.

**Q32 | Statement-based | Thread Life Cycle**

**Consider the statements:**
**I. After its sleep time ends, a thread immediately resumes execution on the CPU.**
**II. A Running thread returns to Runnable when its time slice expires.**
**III. A Runnable thread becomes Running when the scheduler allocates the CPU.**
**Which statements are correct?**

**A.** II and III only

**B.** I and II only

**C.** I and III only

**D.** I, II and III

**✓ Ans: A — II and III only**

**Explanation:** After sleep ends the thread goes to Runnable and must wait to be scheduled again, so I is false.

**Q33 | Transition | Thread Life Cycle**

**A thread is waiting for an I/O operation. When the operation completes, the thread moves:**

**A.** From Blocked/Waiting to Terminated

**B.** From Blocked/Waiting directly to Running

**C.** From Blocked/Waiting to Runnable

**D.** From Blocked/Waiting back to New

**✓ Ans: C — From Blocked/Waiting to Runnable**

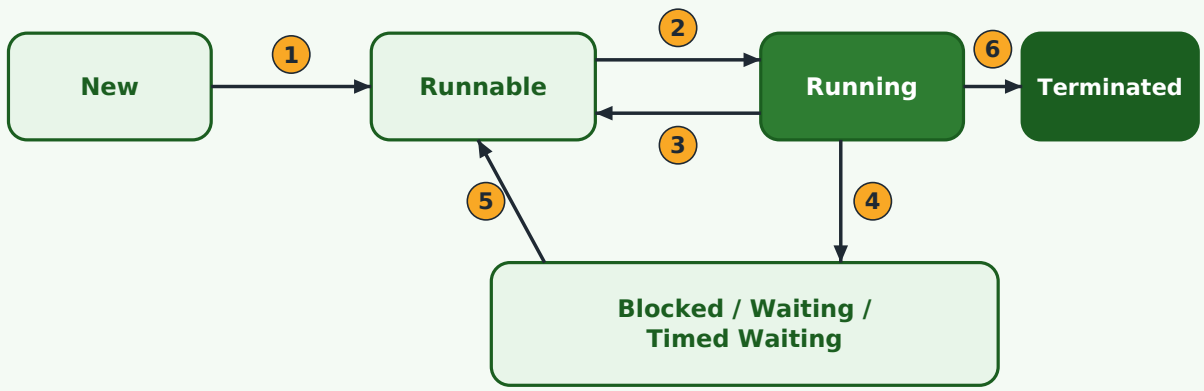
**Explanation:** It rejoins the ready pool and must be scheduled before it runs again.

SECTION 4

## Scenario / Diagram-Based Questions

8 Questions

Life-cycle diagram and applied scenarios




*Arrows are numbered 1–6; the events behind them are to be identified.*

**Figure:** Thread life-cycle diagram used for Questions Q34–Q37.

**Q34 | Diagram | Thread Life Cycle**

**In the life-cycle diagram above, arrow 1 (New → Runnable) is triggered by:**

**A.** Invoking start() on the thread

**B.** Completion of an I/O request

**C.** Completion of the run() method

**D.** Expiry of the time slice

**✓ Ans: A — Invoking start() on the thread**

**Explanation:** start() makes a New thread Runnable.

**Q35 | Diagram | Thread Life Cycle**

**Arrow 3 (Running → Runnable) typically occurs when:**

**A.** start() is invoked on the thread

**B.** The run() method finishes execution

**C.** The running thread's time slice expires

**D.** The thread requests an I/O operation

**✓ Ans: C — The running thread's time slice expires**

**Explanation:** On time-slice expiry the thread returns to the ready pool and waits for the CPU again.

**Q36 | Diagram | Thread Life Cycle**

**Which transition is deliberately absent from the diagram because it is not allowed?**

**A.** New → Runnable via start()

**B.** Running → Terminated normally

**C.** Blocked/Waiting → Running directly

**D.** Runnable → Running by scheduler

**✔ Ans: C — Blocked/Waiting → Running directly**

**Explanation:** A waiting thread must first become Runnable (arrow 5) before it can be dispatched.

**Q37 | Diagram | Thread Life Cycle**

**How many arrows in the diagram lead into the Runnable state?**

**A.** One

**B.** Four

**C.** Two

**D.** Three

**✔ Ans: D — Three**

**Explanation:** Arrows 1 (from New), 3 (from Running) and 5 (from Blocked/Waiting).

**Q38 | Scenario | Thread Life Cycle**

**On a single-core CPU, four started threads exist. At any one instant, how many can be in the Running state?**

**A.** Exactly one

**B.** Four

**C.** None, unless they are in the New state

**D.** Two

**✔ Ans: A — Exactly one**

**Explanation:** A single core executes one thread at a time; the others wait as Runnable or non-runnable.

**Q39 | Scenario | Thread Life Cycle**

**Which sequence is a valid path through the life cycle of a thread?**

**A.** Runnable → New → Running → Blocked → Runnable → Terminated

**B.** New → Blocked → Runnable → Running → Terminated → New

**C.** New → Runnable → Running → Blocked → Runnable → Running → Terminated

**D.** New → Running → Runnable → Blocked → Terminated → Runnable

**✔ Ans: C — New → Runnable → Running → Blocked → Runnable → Running → Terminated**

**Explanation:** Every thread starts in New, passes through Runnable before Running, and ends in Terminated.

**Q40 | Scenario | Thread Life Cycle**

**A program has three thread objects t1, t2 and t3. start() is invoked on t1 and t2 only. How many threads are still in the New state?**

**A.** Zero

**B.** Two

**C.** Three

**D.** One

**✔ Ans: D — One**

**Explanation:** t1 and t2 move to Runnable; only t3 remains New.

**Q41 | Scenario | Thread Life Cycle**

**At some instant, thread t1 is in Timed Waiting and thread t2 is Runnable. Which thread can the scheduler allocate the CPU to?**

**A.** Only t1

**B.** Only t2

**C.** Neither t1 nor t2

**D.** Both t1 and t2

**✔ Ans: B — Only t2**

**Explanation:** Only Runnable threads are eligible for the CPU; a Timed Waiting thread must first become Runnable.

SECTION 5

## Important & Tricky Questions

8 Questions

Statement, Assertion–Reason and Match-the-Following

**Q42 | Tricky | Thread Life Cycle**

**A thread has completed its run() method. What happens if start() is invoked on it again?**

**A.** An IllegalThreadStateException is thrown

**B.** The thread resumes in the Runnable state

**C.** The thread restarts from the New state

**D.** The call is silently ignored with no effect

**✔ Ans: A — An IllegalThreadStateException is thrown**

**Explanation:** A Terminated thread can never be restarted; a fresh thread object is required.

**Q43 | Tricky | Thread Life Cycle**

**In Java's *Thread.State* enum, the classic "Running" state is represented by:**

**A.** ACTIVE

**B.** RUNNING

**C.** EXECUTING

**D.** RUNNABLE

**✔ Ans: D — RUNNABLE**

**Explanation:** Java has no separate RUNNING constant; RUNNABLE covers both ready and running threads.

**Q44 | Tricky | Thread Life Cycle**

**run() is called directly on a thread object instead of start(). What is the state of that thread object?**

**A.** It stays New; run() simply executes as an ordinary method call

**B.** It moves to Timed Waiting for a while

**C.** It becomes Terminated immediately after the call

**D.** It becomes Runnable and runs in a new thread

**✔ Ans: A — It stays New; run() simply executes as an ordinary method call**

**Explanation:** Only start() moves a thread out of the New state.

**Q45 | Statement-based | Thread Life Cycle**

**Consider the statements:**
**I. A thread in the New state is not alive.**
**II. A thread in the Terminated state is not alive.**
**III. A thread in the Runnable state is alive.**
**IV. A thread in the Timed Waiting state is dead.**
**Which statements are correct?**

**A.** I, II and III only
**B.** I, III and IV only
**C.** II, III and IV only
**D.** I, II, III and IV

**✔ Ans: A — I, II and III only**

**Explanation:** A thread is alive from start() until termination; Timed Waiting threads are alive.

**Q46 | Assertion-Reason | Multithreading**

**Assertion (A): A Runnable thread is always executing instructions on the CPU.**
**Reason (R): In Java, the RUNNABLE state covers both threads that are running and threads that are ready to run.**

**A.** Both A and R are true, but R is not the correct explanation of A
**B.** A is true, but R is false
**C.** Both A and R are true, and R is the correct explanation of A
**D.** A is false, but R is true

**✔ Ans: D — A is false, but R is true**

**Explanation:** A Runnable thread may be waiting in the ready pool, so A is false; R is true.

**Q47 | Assertion-Reason | Multithreading**

**Assertion (A): A thread moves from New to Runnable when start() is invoked.**
**Reason (R): A newly created thread moves directly to the Running state.**

**A.** A is true, but R is false
**B.** Both A and R are true, but R is not the correct explanation of A
**C.** Both A and R are true, and R is the correct explanation of A
**D.** A is false, but R is true

**✔ Ans: A — A is true, but R is false**

**Explanation:** A is true, but a new thread enters Runnable first, never Running directly, so R is false.

**Q48 | Assertion–Reason | Multithreading**

**Assertion (A): A terminated thread cannot be restarted.**
**Reason (R): The Terminated state means the thread has completed its execution and its life cycle has ended.**

**A.** A is false, but R is true

**B.** Both A and R are true, and R is the correct explanation of A

**C.** Both A and R are true, but R is not the correct explanation of A

**D.** A is true, but R is false

**✔ Ans: B — Both A and R are true, and R is the correct explanation of A**

**Explanation:** Because the life cycle is over, the thread cannot be revived; R explains A.

**Q49 | Match | Thread Life Cycle**

**Match the state with its meaning:**
**State: 1. New 2. Runnable 3. Timed Waiting 4. Terminated**
**Meaning: a. run() has completed b. Ready for the CPU or running c. start() not yet invoked d. Waiting for a fixed period**

**A.** 1-b, 2-c, 3-d, 4-a

**B.** 1-c, 2-b, 3-d, 4-a

**C.** 1-a, 2-b, 3-d, 4-c

**D.** 1-c, 2-b, 3-a, 4-d

**✔ Ans: B — 1-c, 2-b, 3-d, 4-a**

**Explanation:** New = not started (c); Runnable = ready/running (b); Timed Waiting = fixed wait (d); Terminated = finished (a).

SECTION 6

## Mixed OAVS TGT Practice Set

10 Questions

Exam-style revision across both topics

**Q50 | Mixed | Thread Life Cycle**

**Which of the following is NOT a state in Java's *Thread.State* enum?**

**A.** Suspended

**B.** Terminated

**C.** Runnable

**D.** New

**✔ Ans: A — Suspended**

**Explanation:** Java defines NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING and TERMINATED; there is no SUSPENDED.

**Q51 | Scenario | Multithreading**

**A web server must serve many clients at the same time. Which approach suits this requirement best?**

**A.** Rejecting a new request until the previous one has finished

**B.** Processing requests strictly one after another in a single thread

**C.** Handling each client request in a separate thread

**D.** Compiling the server code again for every client

**✔ Ans: C — Handling each client request in a separate thread**

**Explanation:** Multithreading lets many requests make progress concurrently and keeps the server responsive.

**Q52 | Tricky | Thread Life Cycle**

**Which two states are visited at most once during the life of a thread?**

**A.** Waiting and Timed Waiting

**B.** Runnable and Blocked

**C.** Runnable and Terminated

**D.** New and Terminated

**✔ Ans: D — New and Terminated**

**Explanation:** A thread is New only at creation and Terminated only at the end; Runnable and waiting states may repeat.

**Q53 | Statement-based | Thread Life Cycle**

**Which statement about a thread in the Blocked/Waiting state is correct?**

**A.** It does not consume CPU time until it becomes Runnable again

**B.** It moves back to New when its wait ends

**C.** It is automatically Terminated after one second of waiting

**D.** It uses complete time slices continuously while waiting

**✔ Ans: A — It does not consume CPU time until it becomes Runnable again**

**Explanation:** Non-runnable threads are not scheduled, so they use no CPU time.

**Q54 | Mixed | Thread Life Cycle**

**Which pair of state and meaning is correctly matched?**

**A.** Runnable – run() has already completed

**B.** Terminated – waiting for an I/O operation to finish

**C.** Timed Waiting – sleeping for a fixed duration

**D.** New – start() has already been invoked on it

**✔ Ans: C — Timed Waiting – sleeping for a fixed duration**

**Explanation:** New means not yet started; Terminated means finished; Runnable means ready or running.

**Q55 | Assertion–Reason | Multithreading**

**Assertion (A): A Terminated thread cannot be started again.**
**Reason (R): A Runnable thread may be waiting for the CPU to be allocated to it.**

**A.** A is false, but R is true

**B.** A is true, but R is false

**C.** Both A and R are true, and R is the correct explanation of A

**D.** Both A and R are true, but R is not the correct explanation of A

**✔ Ans: D — Both A and R are true, but R is not the correct explanation of A**

**Explanation:** Both statements are true, but the second does not explain why a terminated thread cannot restart.

**Q56 | Tricky | Thread Life Cycle**

**When the first statement of main() begins executing, the main thread is in which state?**

**A.** Terminated

**B.** Runnable

**C.** Timed Waiting

**D.** New

**✔ Ans: B — Runnable**

**Explanation:** The main thread is already started and executing; in Java this is the RUNNABLE state.

**Q57 | Tricky | Thread Life Cycle**

**What is the minimum number of distinct *Thread.State* values a thread must pass through if it is started and runs to completion?**

**A.** Five

**B.** Three

**C.** Two

**D.** Four

**✔ Ans: B — Three**

**Explanation:** At least NEW, RUNNABLE and TERMINATED.

**Q58 | Concept | Thread Life Cycle**

**A Runnable thread that is not currently on the CPU is waiting for:**

**A.** Arrival of input from the user at the keyboard

**B.** Expiry of a sleep period of fixed length

**C.** Completion of its own run() method

**D.** Allocation of the CPU by the thread scheduler

**✓ Ans: D — Allocation of the CPU by the thread scheduler**

**Explanation:** Ready threads queue up until the scheduler dispatches them.

**Q59 | Concept | Multithreading**

**Which statement is TRUE?**

**A.** Multithreading lets a program do useful work while one of its threads waits for I/O

**B.** Terminated threads return to the Runnable state after clean-up is done

**C.** Every thread in a process owns its own separate heap memory area

**D.** A thread in the New state is already eligible for CPU allocation

**✓ Ans: A — Multithreading lets a program do useful work while one of its threads waits for I/O**

**Explanation:** While one thread waits, others keep the CPU busy; this is a key advantage of multithreading.

SECTION 7

## Supplementary Practice Set

21 Questions

Extra coverage: process vs thread, terminology, Java states and getState()

**Q60 | Concept | Multithreading**

**Which is a correct difference between a process and a thread?**

**A.** Switching between processes is lighter than switching between threads

**B.** A process has its own address space, whereas the threads of a process share one

**C.** A process is always created inside a thread of another process

**D.** A thread has its own address space, whereas processes share a common one

**✔ Ans: B — A process has its own address space, whereas the threads of a process share one**

**Explanation:** Process = separate address space and heavier; thread = lightweight and shares the process's memory.

**Q61 | Concept | Multithreading**

**Which statement correctly distinguishes multithreading from multiprocessing?**

**A.** Multithreading runs many threads inside one process; multiprocessing uses several processes or CPUs

**B.** Multithreading can run only on separate machines connected by a network

**C.** Multithreading needs several processes, while multiprocessing needs only a single thread

**D.** Multiprocessing shares heap memory among all its processes by default

**✔ Ans: A — Multithreading runs many threads inside one process; multiprocessing uses several processes or CPUs**

**Explanation:** Multithreading is within a single process; multiprocessing involves multiple processes and/or processors.

**Q62 | Scenario | Multithreading**

**Which application is a typical use of multithreading?**

**A.** A program that adds two numbers once and exits

**B.** A game that handles graphics, sound and user input together

**C.** A program with no input, no output and nothing to wait for

**D.** A one-line script that prints a single message

**✔ Ans: B — A game that handles graphics, sound and user input together**

**Explanation:** Games have several activities that must progress at the same time, which suits threads.

**Q63 | Terminology | Multithreading**

***Throughput* in the context of multithreading means:**

**A.** The time taken by a single thread to start

**B.** The number of tasks completed in a given period of time

**C.** The number of states a thread can pass through

**D.** The amount of memory consumed by each thread

**✔ Ans: B — The number of tasks completed in a given period of time**

**Explanation:** Multithreading can raise throughput by keeping the CPU busy with ready tasks.

**Q64 | Terminology | Multithreading**

**In a Java program, the thread that begins executing the main() method is called the:**

**A.** Child thread

**B.** Scheduler thread

**C.** Main thread

**D.** Parent process

**✓ Ans: C — Main thread**

**Explanation:** The JVM starts the main thread, which runs main(); other threads can be started from it.

**Q65 | Advantage | Multithreading**

**While one thread waits for a slow input/output operation, multithreading allows the CPU to:**

**A.** Terminate the waiting thread at once

**B.** Shut down permanently to save power

**C.** Stay idle until the I/O operation is finished

**D.** Execute another ready thread in the meantime

**✓ Ans: D — Execute another ready thread in the meantime**

**Explanation:** Overlapping waiting with useful work improves CPU utilization.

**Q66 | Terminology | Multithreading**

**A *time slice* is:**

**A.** The delay between creating a thread object and starting it

**B.** The total time a thread needs to complete its whole run() method

**C.** The gap between the terminations of two different threads

**D.** A short, fixed amount of CPU time given to a thread before it may be switched out

**✓ Ans: D — A short, fixed amount of CPU time given to a thread before it may be switched out**

**Explanation:** Time slicing lets many threads share one CPU in turns.

**Q67 | Assertion–Reason | Multithreading**

**Assertion (A): Threads are often called lightweight processes.**
**Reason (R): Threads of a process share its resources, so they need less overhead to create and manage.**

**A.** A is false, but R is true

**B.** A is true, but R is false

**C.** Both A and R are true, and R is the correct explanation of A

**D.** Both A and R are true, but R is not the correct explanation of A

**✓ Ans: C — Both A and R are true, and R is the correct explanation of A**

**Explanation:** Shared resources make threads cheaper than processes, which explains the 'lightweight' label.

**Q68 | Assertion–Reason | Multithreading**

**Assertion (A): Concurrency always means that tasks run at exactly the same instant.**
**Reason (R): Concurrent tasks may simply be interleaved on a single core through time slicing.**

**A.** A is true, but R is false

**B.** A is false, but R is true

**C.** Both A and R are true, and R is the correct explanation of A

**D.** Both A and R are true, but R is not the correct explanation of A

**✔ Ans: B — A is false, but R is true**

**Explanation:** Concurrency is about overlapping progress, not necessarily simultaneous execution, so A is false.

**Q69 | Match | Multithreading**

**Match List I with List II:**
**List I: 1. Time slice 2. Throughput 3. Main thread 4. Process**
**List II: a. CPU time given to a thread in one turn b. Tasks completed per unit time c. Thread that executes main() d. Program in execution with its own address space**

**A.** 1-b, 2-a, 3-c, 4-d

**B.** 1-c, 2-b, 3-a, 4-d

**C.** 1-a, 2-b, 3-d, 4-c

**D.** 1-a, 2-b, 3-c, 4-d

**✔ Ans: D — 1-a, 2-b, 3-c, 4-d**

**Explanation:** Each term matches its meaning directly.

**Q70 | Life Cycle | Thread Life Cycle**

**The classic five-state model of the thread life cycle consists of:**

**A.** New, Ready, Running, Blocked/Waiting and Dead

**B.** Created, Running, Sleeping, Joined and Closed

**C.** New, Start, Stop, Pause and Dead

**D.** Init, Ready, Active, Suspended and Exit

**✔ Ans: A — New, Ready, Running, Blocked/Waiting and Dead**

**Explanation:** In Java's enum, Ready and Running are combined as RUNNABLE, and Blocked/Waiting is split further.

**Q71 | State | Thread Life Cycle**

**A thread in the BLOCKED state, unlike a RUNNABLE thread, is:**

**A.** Already terminated and removed from memory by the JVM

**B.** Not yet started by the program using start()

**C.** Always executing on one of the available CPU cores

**D.** Not eligible for the CPU until the resource it needs becomes available

**✔ Ans: D — Not eligible for the CPU until the resource it needs becomes available**

**Explanation:** BLOCKED is a non-runnable state; the thread returns to Runnable once it can proceed.

**Q72 | Terminology | Thread Life Cycle**

**Which method of the Thread class returns the current state of a thread?**

**A.** currentState()

**B.** getState()

**C.** isState()

**D.** getStatus()

**✔ Ans: B — getState()**

**Explanation:** getState() returns a value of the Thread.State enumeration.

**Q73 | State | Thread Life Cycle**

**What does getState() return for a thread whose run() method has finished?**

**A.** RUNNABLE

**B.** TERMINATED

**C.** WAITING

**D.** NEW

**✔ Ans: B — TERMINATED**

**Explanation:** After completion, the state is TERMINATED.

**Q74 | Diagram | Thread Life Cycle**

**Arrange the states of a normally completing thread in the correct order:**
**(i) Terminated (ii) Runnable (iii) New (iv) Running**

**A.** (iii), (ii), (iv), (i)

**B.** (ii), (iii), (iv), (i)

**C.** (iii), (iv), (ii), (i)

**D.** (iii), (ii), (i), (iv)

**✔ Ans: A — (iii), (ii), (iv), (i)**

**Explanation:** New → Runnable → Running → Terminated.

**Q75 | Scenario | Thread Life Cycle**

**A thread object exists but start() has not been called. What does its isAlive() method return?**

**A.** true

**B.** It throws an exception

**C.** It depends on the thread scheduler

**D.** false

**✔ Ans: D — false**

**Explanation:** A thread is alive only from start() until it terminates.

**Q76 | State | Thread Life Cycle**

**Which statement about the Dead (Terminated) state is correct?**

**A.** The thread keeps consuming CPU time quietly in the background

**B.** The thread object is erased from memory immediately after it ends

**C.** The thread's execution is over, although its object may still exist in memory

**D.** The thread returns to the Ready state after some clean-up work

**✔ Ans: C — The thread's execution is over, although its object may still exist in memory**

**Explanation:** The object may remain referenced, but the thread can never run again.

**Q77 | Transition | Thread Life Cycle**

**Thread A is Runnable while Thread B is running. For A to start executing, which transitions occur?**

**A.** A moves New → Running directly, skipping the ready pool

**B.** B moves Running → Runnable and A moves Runnable → Running

**C.** B moves to Terminated and A moves to Timed Waiting at once

**D.** A moves Blocked → Running directly without becoming Runnable

**✔ Ans: B — B moves Running → Runnable and A moves Runnable → Running**

**Explanation:** At the end of B's time slice, the scheduler dispatches A.

**Q78 | Diagram | Thread Life Cycle**

**In a life-cycle diagram, the arrow from Running to Blocked/Waiting is labelled with:**

**A.** Expiry of the time slice of the thread

**B.** Completion of the run() method

**C.** An I/O request or a sleep call

**D.** A start() call on the thread object

**✔ Ans: C — An I/O request or a sleep call**

**Explanation:** These events make a running thread give up the CPU and wait.

**Q79 | Tricky | Thread Life Cycle**

**Which statement about the Runnable state is FALSE?**

**A.** It is entered again after a sleep period of the thread ends

**B.** It is entered from New when start() is invoked on the thread

**C.** It may be waiting in the ready pool for the CPU to be allocated

**D.** A Runnable thread is guaranteed to be executing on the CPU at this instant

**✔ Ans: D — A Runnable thread is guaranteed to be executing on the CPU at this instant**

**Explanation:** Runnable means eligible to run, not necessarily running.

**Q80 | Statement-based | Multithreading**

**Consider the statements:**
**I. Concurrency does not require multiple CPU cores.**
**II. Parallelism requires multiple CPU cores.**
**III. Each thread of a process has its own independent heap.**
**Which statements are correct?**

**A.** II and III only

**B.** I and III only

**C.** I, II and III

**D.** I and II only

**✔ Ans: D — I and II only**

**Explanation:** Time slicing gives concurrency on one core; true parallelism needs several cores; threads share the heap, so III is false.

ANSWER KEY

## Quick Answer Key

80 Questions

Q. No. → Correct option

| | | | | | |
|---|---|---|---|---|---|
| **Q1** – C | **Q2** – C | **Q3** – B | **Q4** – D | **Q5** – A | **Q6** – D |
| **Q7** – B | **Q8** – A | **Q9** – B | **Q10** – C | **Q11** – A | **Q12** – D |
| **Q13** – C | **Q14** – C | **Q15** – A | **Q16** – C | **Q17** – B | **Q18** – A |
| **Q19** – D | **Q20** – D | **Q21** – C | **Q22** – C | **Q23** – B | **Q24** – D |
| **Q25** – B | **Q26** – B | **Q27** – C | **Q28** – B | **Q29** – A | **Q30** – A |
| **Q31** – C | **Q32** – A | **Q33** – C | **Q34** – A | **Q35** – C | **Q36** – C |
| **Q37** – D | **Q38** – A | **Q39** – C | **Q40** – D | **Q41** – B | **Q42** – A |
| **Q43** – D | **Q44** – A | **Q45** – A | **Q46** – D | **Q47** – A | **Q48** – B |
| **Q49** – B | **Q50** – A | **Q51** – C | **Q52** – D | **Q53** – A | **Q54** – C |
| **Q55** – D | **Q56** – B | **Q57** – B | **Q58** – D | **Q59** – A | **Q60** – B |
| **Q61** – A | **Q62** – B | **Q63** – B | **Q64** – C | **Q65** – D | **Q66** – D |
| **Q67** – C | **Q68** – B | **Q69** – D | **Q70** – A | **Q71** – D | **Q72** – B |
| **Q73** – B | **Q74** – A | **Q75** – D | **Q76** – C | **Q77** – B | **Q78** – C |
| **Q79** – D | **Q80** – D | | | | |